

[Image]

I built the Order Tracking screen as a single-file mobile page with mock data. I haven't opened it or tested it on a phone, so click through every scenario before you submit.

It covers the three required situations, plus loading and error states. A "Demo scenario" strip at the top switches between them:

- **Delayed:** amber hero with the old estimate struck through and a new one. It shows the reason for the delay, and if it's still late by the new time, the user can request a replacement or refund.
- **Delivered, not received:** red-tinted hero showing when the courier marked it delivered and where it was left. It has a "Before you report" checklist and a "I didn't receive it" button that opens a report form.
- **Tracking not available:** blue preparing state with an expected ship date and delivery window, plus a "Notify me when it ships" toggle. The timeline shows what's done and what's next, so it doesn't look broken.
- **Loading and error:** a skeleton screen while loading, and an error card with "Try again" and "Contact support".

Every state also has:

- A five-step delivery timeline: order placed, processing, shipped, out for delivery, delivered.
- An order summary with expandable order details.
- Contact support (chat, call or email) and report a delivery issue, both as bottom sheets that end in a confirmation.
- A layout that works from about 360 to 430px wide, with light and dark themes.

Still to do for submission: I can't deploy or create a repo for you, so these two are on you.

- Put `order-tracking.html` in a new GitHub repo as `index.html`, with a short README covering setup and run (open the file, or run `npx serve`).
- Deploy it for the live URL. GitHub Pages, Netlify Drop or Vercel all work for a static file, with no build step.

Since the task says React or Next.js is fine but not required, plain HTML works. If you'd prefer a Next.js version to match your usual stack, tell me and I'll convert it.

Page: <https://claude.ai/artifact/1hnbzZ8a4xCiDZjG3e89NC>

i want to build it in react. give me it step by step first build react project then use a component library daisy ui and tailwind css then step by step

Here's the full build using Vite, React, Tailwind v4 and daisyUI 5. The code matches the page I published earlier.

Step 1: Create the React project

```
npm create vite@latest order-tracking -- --template react
cd order-tracking
npm install
```

Step 2: Install Tailwind and daisyUI

```
npm install tailwindcss @tailwindcss/vite daisyui
```

Step 3: Add the Tailwind plugin to Vite

vite.config.js

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'
import tailwindcss from '@tailwindcss/vite'

export default defineConfig({
  plugins: [react(), tailwindcss()],
})
```

Step 4: Enable daisyUI in your CSS

Replace everything in `src/index.css` with:

```
@import "tailwindcss";
@plugin "daisyui";
```

Then delete `src/App.css`. In `index.html`, set `<html lang="en">` and set the `<title>` to "Order tracking".

Step 5: Mock data

Create `src/data/scenarios.js`:

```
export const ORDER = {
  id: '#48213-7790',
  placed: 'Sat, Sep 26',
  addr: 'House 12, Road 5, Rangpur',
  carrier: 'SwiftCourier',
  trk: 'SC-90417723',
  pay: 'Card ending 4417',
}
```

```

items: [
  { n: 'Trail Runner Jacket', v: 'Olive, M', q: 1, p: 89, e: '👕' },
  { n: 'Merino Socks (3-pack)', v: 'Grey', q: 1, p: 24, e: '🧦' },
],
]
}

export const STEPS = ['Order placed', 'Processing', 'Shipped', 'Out for delivery', 'Delivered']

// Full class names so Tailwind can detect them
export const TONE = {
  warning: { box: 'bg-warning/10 border-warning/30', badge: 'badge-warning', btn: 'btn-warning', dot: 'border-warning text-warning ring-warning/20' },
  error: { box: 'bg-error/10 border-error/30', badge: 'badge-error', btn: 'btn-error', dot: 'border-error text-error ring-error/20' },
  info: { box: 'bg-info/10 border-info/30', badge: 'badge-info', btn: 'btn-info', dot: 'border-info text-info ring-info/20' },
}

export const SCENARIOS = {
  delayed: {
    tone: 'warning', pill: 'Delayed', title: 'Running late',
    msg: 'Your package missed its delivery window. It is still on its way and is with the courier.',
    eta: ['Was due', 'Today, 6:00 PM', 'New estimate', 'Tomorrow, by 2 PM'],
    cur: 3,
    stamps: ['Sep 26, 4:10 PM', 'Sep 26, 9:30 PM', 'Sep 27, 11:05 AM', 'Today, 8:40 AM', ''],
    notes: { 3: 'Delayed by heavy traffic at the local hub' },
    alert: '🚚 Sorry for the delay. If it has not arrived by tomorrow, 2 PM, you can request a replacement or refund.',
    primary: ['Contact support', 'support'],
    secondary: ['Report a delivery issue', 'report'],
  },
  missing: {
    tone: 'error', pill: 'Marked delivered', title: 'Delivered, but not received?',
    msg: 'The courier marked this order as delivered today at

```

```

1:42 PM. If you do not have it, we will help you find it.',
  eta: ['Delivered', 'Today, 1:42 PM', 'Left at', 'Front
door'],
  cur: 4, missing: true,
  stamps: ['Sep 24, 4:10 PM', 'Sep 24, 9:30 PM', 'Sep 25, 11:05
AM', 'Today, 9:15 AM', 'Today, 1:42 PM'],
  notes: { 4: 'Left at front door · photo on file' },
  alert: '🔴 Delivery confirmed near House 12, Road 5. Check
with neighbors and any safe place before reporting.',
  primary: ['I didn't receive it', 'missing'],
  secondary: ['Contact support', 'support'],
},
pending: {
  tone: 'info', pill: 'Preparing', title: 'Tracking starts
soon',
  msg: 'The seller is packing your order. Tracking appears here
once the courier scans your package.',
  eta: ['Expected to ship', 'By Sep 29', 'Estimated delivery',
'Oct 1 – Oct 3'],
  cur: 1, pending: true,
  stamps: ['Today, 10:20 AM', 'In progress', '', '', ''],
  notes: { 1: 'Usually takes up to 24 hours' },
  alert: '🔔 We'll notify you as soon as a tracking number is
assigned.',
  primary: ['Notify me when it ships', 'notify'],
  secondary: ['Contact support', 'support'],
},
}

```

Step 6: Components

Create the folder `src/components/`.

StatusHero.jsx

```

import { TONE } from '../data/scenarios'

export default function StatusHero({ s, notified, onAction }) {
  const t = TONE[s.tone]
  const late = s.tone === 'warning'
  return (
    <section className={`rounded-2xl border p-5 ${t.box}`} aria-
labelledby="status-title">
      <span className={`badge badge-soft
${t.badge}`}>{s.pill}</span>

```

```

    <h1 id="status-title" className="mt-3 text-2xl font-
bold">{s.title}</h1>
    <p className="mt-1 opacity-80">{s.msg}</p>

    <div className="mt-4 flex justify-between gap-3 border-t
border-base-content/10 pt-3 text-sm">
      <div>
        <div className="opacity-60">{s.eta[0]}</div>
        <div className={`font-semibold ${late ? 'line-through
opacity-60' : ''}`}>{s.eta[1]}</div>
      </div>
      <div className="text-right">
        <div className="opacity-60">{s.eta[2]}</div>
        <div className="font-semibold">{s.eta[3]}</div>
      </div>
    </div>

    <div role="alert" className="alert mt-4 text-
sm"><span>{s.alert}</span></div>

    <div className="mt-4 flex flex-wrap gap-2">
      <button className={`btn flex-1 ${t.btn}`} onClick={() =>
onAction(s.primary[1])}>
        {s.pending && notified ? '✓ You'll be notified' :
s.primary[0]}
      </button>
      <button className="btn flex-1" onClick={() =>
onAction(s.secondary[1])}>{s.secondary[0]}</button>
    </div>
  </section>
)
}

```

Timeline.jsx

```

import { STEPS, TONE } from '../data/scenarios'

export default function Timeline({ s, carrier, trk }) {
  const t = TONE[s.tone]
  return (
    <section className="card card-border bg-base-100 mt-4">
      <div className="card-body">
        <h2 className="card-title">Delivery progress</h2>
        <ol aria-label="Delivery progress">
          {STEPS.map((name, i) => {

```

```

    const done = i < s.cur || (s.missing && i === s.cur)
    const now = i === s.cur && !s.missing
    const last = i === STEPS.length - 1
    return (
      <li key={name} className="relative grid grid-cols-[28px_1fr] gap-3 pb-5 last:pb-0">
        {!last && (
          <span className={`absolute left-[13px] top-7 -bottom-0.5 w-0.5 ${done ? 'bg-success' : 'bg-base-300'}`} />
        )}
        <span className={`grid size-7 place-items-center rounded-full border-2 text-xs font-bold ${done ? 'bg-success border-success text-success-content' : now ? `ring-4 ${t.dot}` : 'border-base-300 text-base-content/50'}`}>
          {done ? '✓' : i + 1}
        </span>
        <div>
          <p className={`leading-tight ${!done && !now ? 'opacity-60' : 'font-semibold'}`}>{name}</p>
          {s.notes[i] && <p className="text-sm opacity-70">{s.notes[i]}</p>}
          <p className="text-sm opacity-60">{s.stamps[i] || (i > s.cur ? 'Not started' : '')}</p>
        </div>
      </li>
    )
  )}
</ol>
    {!s.pending && <p className="text-xs opacity-60">{carrier} · Tracking {trk}</p>}
  </div>
</section>
)
}

```

OrderSummary.jsx

```

export default function OrderSummary({ order }) {
  const total = order.items.reduce((a, i) => a + i.p * i.q, 0)
  return (
    <section className="card card-border bg-base-100 mt-4">
      <div className="card-body">
        <h2 className="card-title">Order summary</h2>

```

```

        {order.items.map((i) => (
          <div key={i.n} className="flex items-center gap-3">
            <span className="grid size-12 place-items-center
rounded-lg bg-base-200 text-2xl" aria-hidden="true">{i.e}</span>
            <div className="flex-1">
              <p className="font-semibold">{i.n}</p>
              <p className="text-sm opacity-60">{i.v} · Qty
{ i.q}</p>
            </div>
            <p className="font-semibold">${(i.p *
i.q).toFixed(2)}</p>
          </div>
        ))}
        <div className="flex justify-between border-t border-
base-300 pt-3 font-bold">
          <span>Total</span><span>${total.toFixed(2)}</span>
        </div>

        <div className="collapse collapse-arrow bg-base-200">
          <input type="checkbox" aria-label="Toggle order
details" />
          <div className="collapse-title font-semibold">View
order details</div>
          <div className="collapse-content grid grid-cols-
[90px_1fr] gap-y-1 text-sm">
            <span
className="opacity-60">Order</span><span>{order.id}</span>
            <span
className="opacity-60">Placed</span><span>{order.placed}</span>
            <span className="opacity-60">Ship
to</span><span>{order.addr}</span>
            <span
className="opacity-60">Payment</span><span>{order.pay}</span>
          </div>
        </div>
      </div>
    </section>
  )
}

```

States.jsx (loading and error)

```

export function LoadingState() {
  return (
    <div aria-busy="true" aria-label="Loading tracking">

```

```

        <div className="skeleton h-56 w-full" />
        <div className="skeleton mt-4 h-72 w-full" />
        <div className="skeleton mt-4 h-32 w-full" />
    </div>
  )
}

export function ErrorState({ onRetry, onSupport }) {
  return (
    <div className="card card-border bg-base-100 text-center">
      <div className="card-body items-center">
        <div className="text-4xl">🚚</div>
        <h2 className="card-title">Can't load tracking</h2>
        <p className="opacity-70">We couldn't reach the tracking
service. Your order is safe. Check your connection and try
again.</p>
        <div className="card-actions mt-2">
          <button className="btn btn-primary"
onClick={onRetry}>Try again</button>
          <button className="btn" onClick={onSupport}>Contact
support</button>
        </div>
      </div>
    </div>
  )
}

```

ActionModal.jsx (support, report issue, report missing package)

```

import { useState } from 'react'

const REASONS = {
  missing: [
    ['Not at my address', 'Package isn't at the door or with
neighbors'],
    ['Wrong address', 'Courier went to a different place'],
    ['Package looks stolen', 'Someone may have taken it'],
  ],
  report: [
    ['Order is late', 'It has passed the estimated time'],
    ['Package damaged', 'Box or item arrived damaged'],
    ['Wrong address', 'Courier has the wrong location'],
  ],
}

```

```

export default function ActionModal({ kind, onClose, onToast }) {
  const [reason, setReason] = useState(0)
  const [phase, setPhase] = useState('form') // form | sending |
done
  if (!kind) return null

  const submit = () => {
    setPhase('sending')
    setTimeout(() => setPhase('done'), 800)
  }
  const pick = (msg) => { onClose(); onToast(msg) }

  return (
    <dialog className="modal modal-open modal-bottom sm:modal-
middle" role="dialog" aria-modal="true">
      <div className="modal-box">
        {kind === 'support' && (
          <>
            <h3 className="text-lg font-bold">Contact
support</h3>
            <p className="py-2 opacity-70">Pick the fastest way
to reach us.</p>
            <div className="flex flex-col gap-2">
              <button className="btn btn-outline justify-start"
onClick={() => pick('Opening live chat...')}>💬 Live chat · replies
in ~2 min</button>
              <button className="btn btn-outline justify-start"
onClick={() => pick('Calling support...')}>☎ Call us · 8 AM – 10
PM</button>
              <button className="btn btn-outline justify-start"
onClick={() => pick('Email draft opened')}>✉ Email · reply
within 24 h</button>
            </div>
            <div className="modal-action"><button className="btn"
onClick={onClose}>Close</button></div>
          </>
        )}

        {kind !== 'support' && phase !== 'done' && (
          <>
            <h3 className="text-lg font-bold">{kind === 'missing'
? 'Report missing package' : 'Report a delivery issue'}</h3>
            <p className="py-2 opacity-70">Tell us what happened.
We'll contact the courier for you.</p>
          </>
        )}
      </div>
    </dialog>
  )
}

```

```

        <div className="flex flex-col gap-2">
          {REASONS[kind].map(([t, d], i) => (
            <label key={t} className={`flex cursor-pointer
items-center gap-3 rounded-lg border p-3 ${reason === i ?
'border-primary bg-primary/10' : 'border-base-300'} `}>
              <input type="radio" name="reason"
className="radio radio-primary" checked={reason === i}
onChange={() => setReason(i)} />
              <span><b>{t}</b><br /><small
className="opacity-60">{d}</small></span>
            </label>
          ))}
        </div>
        <textarea className="textarea mt-3 w-full"
placeholder="Add details (optional)" />
        <div className="modal-action">
          <button className="btn"
onClick={onClose}>Cancel</button>
          <button className="btn btn-primary"
onClick={submit} disabled={phase === 'sending'}>
            {phase === 'sending' ? <span className="loading
loading-spinner" /> : 'Submit report'}
          </button>
        </div>
      </>
    )}

    {kind !== 'support' && phase === 'done' && (
      <div className="text-center">
        <div className="text-4xl">✅</div>
        <h3 className="mt-2 text-lg font-bold">Report
received</h3>
        <p className="py-2 opacity-70">Case #RP-20931 is
open. We'll email you within 24 hours with a replacement or
refund.</p>
        <button className="btn btn-primary w-full"
onClick={onClose}>Done</button>
      </div>
    )}
  </div>
  <div className="modal-backdrop" onClick={onClose} />
</dialog>
)
}

```

Step 7: Wire it together in `App.jsx`

Replace `src/App.jsx`:

```
import { useEffect, useState } from 'react'
import { ORDER, SCENARIOS } from '../data/scenarios'
import StatusHero from '../components/StatusHero'
import Timeline from '../components/Timeline'
import OrderSummary from '../components/OrderSummary'
import ActionModal from '../components/ActionModal'
import { LoadingState, ErrorState } from '../components/States'

const DEMO = [
  ['delayed', 'Delayed'],
  ['missing', 'Delivered, not received'],
  ['pending', 'Tracking not available'],
  ['loading', 'Loading'],
  ['error', 'Error'],
]

export default function App() {
  const [view, setView] = useState('delayed')
  const [loading, setLoading] = useState(true)
  const [modal, setModal] = useState(null)
  const [notified, setNotified] = useState(false)
  const [toast, setToast] = useState('')

  // Fake network delay whenever a data scenario is chosen
  useEffect(() => {
    if (!SCENARIOS[view]) return
    setLoading(true)
    const id = setTimeout(() => setLoading(false), 700)
    return () => clearTimeout(id)
  }, [view])

  const showToast = (msg) => {
    setToast(msg)
    setTimeout(() => setToast(''), 2600)
  }

  const onAction = (a) => {
    if (a === 'notify') {
      setNotified((n) => !n)
      showToast(notified ? 'Notification turned off' : 'We'll

```

```

notify you when tracking is available')
    } else setModal(a)
  }

  const s = SCENARIOS[view]

  return (
    <main className="mx-auto min-h-screen max-w-[430px] bg-
base-200 px-4 pb-10 pt-3">
      <p className="text-xs opacity-60">Demo scenario</p>
      <div className="mb-3 flex gap-1.5 overflow-x-auto py-1"
role="group" aria-label="Demo scenario">
        {DEMO.map(([k, l]) => (
          <button key={k} onClick={() => setView(k)} aria-
pressed={view === k}
          className={`btn btn-xs whitespace-nowrap rounded-full
${view === k ? 'btn-neutral' : 'btn-outline'}}>
            {l}
          </button>
        ))}
      </div>

      <header className="mb-3">
        <p className="text-sm opacity-60">Order {ORDER.id}</p>
        <h2 className="text-xl font-bold">Track order</h2>
      </header>

      {view === 'error' ? (
        <ErrorState onRetry={() => setView('delayed')}
onSupport={() => setModal('support')} />
      ) : view === 'loading' || loading ? (
        <LoadingState />
      ) : (
        <>
          <StatusHero s={s} notified={notified}
onAction={onAction} />
          <Timeline s={s} carrier={ORDER.carrier} trk={ORDER.trk}
/>
          {s.missing && (
            <section className="card card-border bg-base-100
mt-4">
              <div className="card-body">
                <h2 className="card-title">Before you report</h2>
                <ul className="list-disc pl-5 text-sm

```

```

opacity-70">
    <li>Check your front door, porch and gate.</li>
    <li>Ask neighbors or building reception.</li>
    <li>Look for a delivery notice from the
courier.</li>
    <li>Wait until end of day; scans sometimes post
early.</li>
    </ul>
  </div>
</section>
)}
<OrderSummary order={ORDER} />
<section className="card card-border bg-base-100 mt-4">
  <div className="card-body">
    <h2 className="card-title">Need help?</h2>
    <p className="text-sm opacity-70">Our team replies
within a few minutes, 8 AM – 10 PM daily.</p>
    <div className="card-actions">
      <button className="btn flex-1" onClick={() =>
setModal('support')}>Contact support</button>
      <button className="btn flex-1" onClick={() =>
setModal('report')}>Report a delivery issue</button>
    </div>
  </div>
</section>
</>
)}

  <ActionModal key={modal} kind={modal} onClose={() =>
setModal(null)} onToast={showToast} />
  {toast && (
    <div className="toast toast-center">
      <div className="alert alert-neutral"
role="status">{toast}</div>
    </div>
  )}
</main>
)
}

```

src/main.jsx from the Vite template already works. Just make sure it imports ./index.css.

Step 8: Run and check

```
npm run dev
```

Open the URL, then use DevTools device mode at 360px and 430px. Click through each demo tab, open both modals, submit a report, and expand "View order details".

Step 9: GitHub and deploy

```
git init
git add .
git commit -m "Order tracking screen"
# create an empty repo on github.com, then:
git remote add origin https://github.com/<you>/order-tracking.git
git branch -M main
git push -u origin main
```

For the live URL, the easiest option is Vercel:

- Go to vercel.com and choose Add New, then Project.
- Import the repo. It detects Vite automatically, so leave the defaults and click Deploy.
- Copy the `*.vercel.app` URL. The task requires a live URL that works without running the project locally, so open it in a private window to confirm.

Step 10: README (required by the task)

Put this in `README.md`:

```
# Order Tracking Screen

Mobile order tracking UI built with React (Vite), Tailwind CSS v4
and daisyUI.

## Run locally
```bash
npm install
npm run dev
```

## Build
```bash
npm run build && npm run preview
```

```

Notes

- Mock data lives in `src/data/scenarios.js`; no backend needed.
- Use the "Demo scenario" tabs at the top to switch between: Delayed, Delivered but not received, Tracking not available, Loading and Error states.
- Designed for 360–430px widths.

Live demo: <your vercel url>

Tell me if you hit an error at any step, and paste the message.

Error: Calling setState synchronously within an effect can trigger cascading renders

Effects are intended to synchronize state between React and external systems such as manually updating the DOM, state management libraries, or other platform APIs. In general, the body of an effect should do one or both of the following:

- * Update external systems with the latest state from React.
- * Subscribe for updates from some external system, calling setState in a callback function when external state changes.

Calling setState synchronously within an effect body causes cascading renders that can hurt performance, and is not recommended. (<https://react.dev/learn/you-might-not-need-an-effect>).

```
25 | useEffect(() => {
26 |   if (!SCENARIOS[view]) return;
    > 27 |   setLoading(true);
    |   ^^^^^^^^^^^^^^^ Avoid calling setState() directly within an effect
28 |   const id = setTimeout(() => setLoading(false), 700);
29 |   return () => clearTimeout(id);
    30 | }, [view]);
```

This error comes from calling `setLoading(true)` inside the effect body. Move that call into the click handler and leave the effect to run only the timer. The `setLoading(false)` inside `setTimeout` is fine, because it runs in a callback.

1. Replace the effect block (and add the `select` function above it):

```
// Choose a scenario; data scenarios start in the loading state
const select = (k) => {
  setView(k);
  setLoading(Boolean(SCENARIOS[k]));
}
```

```
};

// Fake network delay: the effect only runs the timer
useEffect(() => {
  if (!loading) return;
  const id = setTimeout(() => setLoading(false), 700);
  return () => clearTimeout(id);
}, [loading, view]);
```

2. Use `select` instead of `setView` in the two places that change the scenario:

```
onClick={() => select(k)}
```

```
<ErrorState
  onRetry={() => select("delayed")}
  onSupport={() => setModal("support")}
/>
```

Everything else in `App.jsx` stays the same.

How it works:

- `loading` starts as `true`, so the first render shows the skeleton and the timer clears it after 700ms.
- Clicking a scenario tab sets both `view` and `loading` in one event handler, which React batches into a single render.
- Clicking the Loading or Error tabs sets `loading` to `false`, because `SCENARIOS` has no entry for them. Those views are chosen by the `view` checks in the JSX.
- Having `view` in the effect's dependencies restarts the timer if you switch tabs while it is still loading.

If another lint or runtime error appears, paste it and I'll fix that too.

surge deploy

Surge deploys the built `dist` folder, so build first and then publish it.

1. Install Surge

```
npm install -g surge
```

2. Build the project

```
npm run build
```

This creates the `dist/` folder. If the build fails, fix that first, since Surge only uploads what's in `dist`.

3. Deploy

```
surge ./dist
```

The first time, Surge asks you to create an account with an email and password. Then it shows:

- **project:** the path (`./dist`), so press Enter.
- **domain:** it suggests a random name. Type your own, for example `order-tracking-nir.surge.sh`, since a readable URL looks better in a submission.

When it finishes it prints `Success! - Published to order-tracking-nir.surge.sh`.

4. Check the live URL

Open it in a private window and test at 360px and 430px in DevTools. Click through all five demo tabs and both modals.

5. Redeploy after changes

Use the same domain so the URL stays the same:

```
npm run build && surge ./dist order-tracking-nir.surge.sh
```

To save typing, add this to `package.json` under `"scripts"`:

```
"deploy": "npm run build && surge ./dist order-tracking-nir.surge.sh"
```

Then run `npm run deploy` each time.

6. Update the README and push

Put the live URL in the README's "Live demo" line, then commit and push:

```
git add .
git commit -m "Add live demo URL"
git push
```

Make sure `dist` and `node_modules` are in `.gitignore`. The Vite template already includes both.

For the form, submit the `https://order-tracking-nir.surge.sh` URL and your GitHub repo URL.

If you hit a problem, such as "domain belongs to someone else" (pick another name) or a build error, paste the message and I'll help.

give me readme file here Fahim Faysal Nirjhar Full stack developer and add all the important features that a readme file needed

The README is ready. Before you push it, replace these placeholders:

- `YOUR-DOMAIN.surge.sh` with your Surge URL (it appears in the top links and the Deployment section)
- `YOUR-USERNAME` with your GitHub username (Repository link, clone command and Author section)

- The LinkedIn and email lines in the Author section, or delete them if you'd rather not show them

Two more checks:

- **npm run lint:** the Scripts table lists it because the Vite React template includes it. Remove the row if your `package.json` doesn't have that script.
- **Screenshots:** add a couple of images (for example the Delayed and Delivered but not received states) in a `docs/` folder and link them under the Overview. Screenshots make a README look much more finished to an evaluator.



README

MD · Click to download